

Annexes

Transmission vidéo basse latence pour drone FPV

Travail de Bachelor

Non Confidentiel

Départements : TIC

Filière : Informatique et systèmes de communication

Orientation : Systèmes informatiques embarqués

Auteur : Tschantz Nathan

Supervisé par : Favrat Pierre & Mahboob Karimian

Date : 20 juillet 2026

Table des matières

1	Guide de connexion et d'accès aux équipements	5
1.1	Accès au microcontrôleur d'émission (STM32 / EKH05)	5
1.2	Accès à l'ordinateur d'acquisition (Raspberry Pi 4B)	5
1.3	Accès au routeur relais de réception (GL.iNet MT3000 / OpenWrt)	5
1.4	Accès et adressage des stations de réception (RX)	5
1.5	Tableau récapitulatif d'adressage et d'accès	6
2	Modifications exploratoires du pilote Morse Micro	6
3	Mesure de latence matérielle via bouton partagé	7
3.1	Le Code TX (Émetteur de test : <code>latency_tx.c</code>)	7
3.2	Le Code RX (Chronomètre de précision : <code>latency_rx.c</code>)	9
4	Code de la Gateway finale (Raspberry Pi TX : <code>gateway_spi.c</code>)	11
5	Firmware MCU du transmetteur (STM32U5 / <code>spi_interface.c</code>)	14
6	Système de transmission alternatif DVB-T2 (SDR / BladeRF)	20
6.1	Matériel et architecture	20
6.2	Configuration et pipeline de l'émetteur (Raspberry Pi)	20
6.3	Configuration du récepteur et décodage (PC Windows)	21
6.3.1	Approche standard : VLC Media Player	21
6.3.2	Approche secondaire : TSDuck	22
6.3.3	Pipeline d'affichage GStreamer (Commun aux deux méthodes)	22
6.4	Bilan des mesures	22

Liste des tableaux

- 1 Récapitulatif des accès et du plan d'adressage (Sous-réseau FPV : 192.168.12.x) 6

1 Guide de connexion et d'accès aux équipements

Durant la phase de développement, le système étant distribué sur plusieurs plateformes matérielles dépourvues d'écran (architecture *headless*), des méthodes de connexion spécifiques ont été mises en place pour interagir avec chaque équipement.

1.1 Accès au microcontrôleur d'émission (STM32 / EKH05)

Le module Morse Micro EKH05 intègre un débogueur ST-LINK natif. La connexion physique s'effectue via un simple câble micro-USB relié à la station de développement (PC Windows). Cette liaison fournit deux interfaces virtuelles simultanées :

- **Le flashage et débogage (SWD)** : Géré de manière transparente par STM32CubeIDE via le serveur OpenOCD.
- **La console de logs (UART)** : Accessible via un émulateur de terminal série (tel que PuTTY). Les paramètres de connexion au port COM virtuel sont systématiquement : Baudrate 115200, 8 bits de données, 1 bit d'arrêt, aucune parité.

Du point de vue du réseau HaLow, le STM32 possède l'adresse IP source statique 192.168.12.2.

1.2 Accès à l'ordinateur d'acquisition (Raspberry Pi 4B)

Pour modifier les scripts de capture vidéo et lancer la gateway sans alourdir le processeur avec une interface graphique, la carte d'acquisition a été configurée pour un accès distant via SSH (*Secure Shell*). La connexion s'effectue depuis le PC de développement via le port Ethernet filaire local.

1.3 Accès au routeur relais de réception (GL.iNet MT3000 / OpenWrt)

Le routeur OpenWrt faisant office de pont réseau (Bridge) est le centre névralgique de la réception au sol. Son adresse IP sur l'interface sans-fil est 192.168.12.1. Son administration s'effectue exclusivement via son interface Ethernet (LAN), qui a été isolée pour éviter tout conflit de routage.

- **Interface Web (LuCI)** : Accessible depuis un navigateur web à l'adresse `http://192.168.12.1`. Elle permet de configurer le mode moniteur ou les paramètres de l'AP sans chiffrement.
- **Ligne de commande** : Accessible via l'interface web (Service → Terminal) ou en SSH, elle permet d'injecter le module noyau (`morse.ko`) et de manager le pont logiciel via la commande `brctl addif br-ahwlan wlan0`.

1.4 Accès et adressage des stations de réception (RX)

Le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x.

Station de diagnostic (PC Windows) : L'interface réseau Ethernet doit être fixée manuellement sur l'adresse 192.168.12.10 (Masque : 255.255.255.0, Passerelle : 192.168.12.1).

Station cible (RPI Compute Module / RPi 4B) : Lors de la réception "Bare-Metal" sous Linux pour les mesures de transit réseau par bouton physique commun, l'interface Ethernet nécessite d'être configurée manuellement sur l'adresse statique suivante : 192.168.12.50.

1.5 Tableau récapitulatif d'adressage et d'accès

Le tableau 1 résume l'ensemble du plan d'adressage et des interfaces de gestion de l'architecture déployée :

Équipement / Rôle	Interface d'accès	Adresse IP FPV	Remarques
STM32 / EKH05 (Modem Émetteur)	USB (UART / SWD)	192.168.12.2	Reçoit les trames SPI de la Raspberry Pi et les émet en UDP Unicast.
Raspberry Pi 4B (Acquisition Vidéo TX)	SSH (Ethernet direct)	<i>Non applicable</i>	Connecté en SPI au modem. Pas d'IP sur le réseau FPV.
GL.iNet MT3000 (Routeur Relais RX)	SSH / Web (LuCI)	192.168.12.1	Interface sans-fil <code>ahwlan</code> pontée.
PC Windows (Station de diag. RX)	Accès local	192.168.12.10	Station d'affichage et de métrologie finale.
RPi Compute Module 4 (Station cible RX)	SSH (Wi-Fi dev)	192.168.12.50	Fixée manuellement sur <code>eth0</code> . Dédiée aux tests par bouton.

TABLE 1 – Récapitulatif des accès et du plan d'adressage (Sous-réseau FPV : 192.168.12.x)

2 Modifications exploratoires du pilote Morse Micro

Cette section résume les modifications opérées lors de la phase d'investigation bas niveau sur le module noyau Linux. Afin de tester le forçage du MCS 1 et l'activation du code correcteur d'erreurs (LDPC) directement au cœur de l'OS du routeur, des modifications ont été testées sur le code source C du pilote noyau (`morse_driver`) :

- `core/command.c` : Surcharge de la structure `morse_cmd_req_set_transmission_rate` pour forcer les bits `req.use_ldpc = 1` et injecter le `mcs_index`.
- `core/usb.c` : Injection de l'appel manuel `morse_cmd_set_fixed_transmission_rate(mors, 1, 1, 0, 0)` au sein de la séquence d'initialisation `morse_usb_probe` lors du montage de l'interface.

comme mentionné dans le rapport ces modification se sont avérées superflue.

3 Mesure de latence matérielle via bouton partagé

3.1 Le Code TX (Émetteur de test : latency_tx.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>
#include <sys/mman.h>

#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"
#define GPIO_BTN 17
#define GPIO_HANDSHAKE 24

volatile unsigned *gpio;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 2) &= ~(7 << 12);
    *(gpio + 1) &= ~(7 << 21);
}

int main() {
    int spi_fd;
    uint32_t speed = 8000000;
    uint8_t bits = 8;
    uint32_t mode = 0;
    setup_gpiomem();

    spi_fd = open(SPI_DEVICE, O_RDWR);
    if (spi_fd < 0) { perror("SPI_open"); return 1; }
    ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
    ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
    ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

    uint8_t buffer[CHUNK_SIZE];
```

```
memset(buffer, 0xAA, CHUNK_SIZE);

struct spi_ioc_transfer tr = {
    .tx_buf = (uint64_t)(uintptr_t)buffer,
    .rx_buf = 0,
    .len = CHUNK_SIZE,
    .speed_hz = speed,
    .bits_per_word = bits,
    .cs_change = 0,
};

int test_step = 0;
while (1) {
    if (test_step < 10) {
        printf("\n>>> [UNITAIRE_%d/10] Appui bouton requis...\n", test_step + 1);
    } else {
        printf("\n>>> [BURST_TEST] Appui bouton requis pour rafale de 10 paquets...\n");
    }

    while ( (*(gpio + 13) & (1 << GPIO_BTN)) == 0 );

    if (test_step < 10) {
        int timeout = 0, stm_ready = 1;
        while ( (*(gpio + 13) & (1 << GPIO_HANDSHAKE)) == 0 ) {
            if (timeout++ > 5000000) { stm_ready = 0; break; }
        }
        if (stm_ready) {
            ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr);
            test_step++;
        }
    } else {
        int paquets_ok = 0;
        for (int i = 0; i < 10; i++) {
            int timeout = 0, stm_ready = 1;
            while ( (*(gpio + 13) & (1 << GPIO_HANDSHAKE)) == 0 ) {
                if (timeout++ > 5000000) { stm_ready = 0; break; }
            }
            if (!stm_ready) break;
            ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr);
            paquets_ok++;
            usleep(100);
        }
        if (paquets_ok == 10) test_step = 0;
    }
}
```



```
        usleep(500000);
    }
    close(spi_fd);
    return 0;
}
```

3.2 Le Code RX (Chronomètre de précision : latency_rx.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <time.h>
#include <pthread.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define GPIO_BTN 17
#define PORT 1337

volatile unsigned *gpio;
volatile uint64_t t_start = 0;
volatile uint64_t t_end = 0;
volatile int packet_count = 0;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 1) &= ~(7 << 21);
}

uint64_t get_time_ns() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

void* udp_listener(void* arg) {
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in servaddr = {0};
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
```

```

servaddr.sin_port = htons(PORT);
bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(
    servaddr));
uint8_t buffer[2000];

while(1) {
    recvfrom(sockfd, (char *)buffer, sizeof(buffer), 0, NULL,
        NULL);
    t_end = get_time_ns();
    packet_count++;
}
return NULL;
}

int main() {
    setup_gpiomem();
    pthread_t thread_id;
    pthread_create(&thread_id, NULL, udp_listener, NULL);
    int test_step = 0;
    double sum_latencies = 0;

    printf(">>>RX_CHRONO_PRET.\n");
    while (1) {
        packet_count = 0;
        while ( (*(gpio + 13) & (1 << GPIO_BTN)) == 0 );
        t_start = get_time_ns();
        int target_packets = (test_step < 10) ? 1 : 10;

        while (packet_count < target_packets) {
            if (get_time_ns() - t_start > 1500000000ULL) {
                test_step = 0;
                sum_latencies = 0;
                break;
            }
        }

        if (packet_count == target_packets) {
            double latency_ms = (t_end - t_start) / 1000000.0;
            if (test_step < 10) {
                printf("Test%d/10|Latence: %.3fms\n",
                    test_step + 1, latency_ms);
                sum_latencies += latency_ms;
                if (test_step == 9) {
                    printf("MOYENNE_UNITAIRE: %.3fms\n",
                        sum_latencies / 10.0);
                }
                test_step++;
            } else {

```

```

        printf("LATENCE_BURST_TOTALE_(10_paquets): %.3f ms\n", latency_ms);
        printf("MOYENNE_EN_FLUX_(Latence_/10) : %.3f ms\n", latency_ms / 10.0);
        test_step = 0;
        sum_latencies = 0;
    }
}
usleep(500000);
}
return 0;
}

```

4 Code de la Gateway finale (Raspberry Pi TX : gateway_spi.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/spi/spidev.h>
#include <sys/mman.h>

#define CHUNK_SIZE 1400
#define SPI_DEVICE "/dev/spidev0.0"
#define GPIO_PIN 24
#define RING_BUFFER_SIZE (1024 * 64)

volatile unsigned *gpio;

void setup_gpiomem() {
    int mem_fd = open("/dev/gpiomem", O_RDWR | O_SYNC);
    if (mem_fd < 0) { perror("Erreur_gpiomem"); exit(1); }
    gpio = (volatile unsigned *)mmap(NULL, 4096, PROT_READ |
        PROT_WRITE, MAP_SHARED, mem_fd, 0);
    close(mem_fd);
    *(gpio + 2) &= ~(7 << 12);
}

int wait_for_stm32_ready() {
    int timeout_counter = 0;
    while ((* (gpio + 13) & (1 << GPIO_PIN)) == 0) {
        timeout_counter++;
    }
}

```

```

        if (timeout_counter > 2000000) { return 0; }
    }
    return 1;
}

int main() {
    int spi_fd;
    FILE *cam_pipe;
    int pipe_fd;
    uint32_t speed = 8000000;
    uint8_t bits = 8;
    uint32_t mode = 0;
    setup_gpiomem();

    spi_fd = open(SPI_DEVICE, O_RDWR);
    if (spi_fd < 0) { perror("SPI open"); return 1; }
    ioctl(spi_fd, SPI_IOC_WR_MODE, &mode);
    ioctl(spi_fd, SPI_IOC_WR_BITS_PER_WORD, &bits);
    ioctl(spi_fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed);

    const char *cmd = "rpicam-vid -t 0 -n -o --width 320 --height
        240 --framerate 60
        --codec jpeg --denoise off --exposure sport
        --metering centre
        --awb daylight --quality 8 --flush";

    printf(">>> Lancement de la capture camera directe...\n");
    cam_pipe = popen(cmd, "r");
    if (!cam_pipe) { close(spi_fd); return 1; }

    pipe_fd = fileno(cam_pipe);
    int flags = fcntl(pipe_fd, F_GETFL, 0);
    fcntl(pipe_fd, F_SETFL, flags | O_NONBLOCK);

    uint8_t spi_buffer[CHUNK_SIZE];
    uint8_t *recv_buf = malloc(RING_BUFFER_SIZE);
    size_t recv_len = 0;
    int drop_counter = 0;

    struct spi_ioc_transfer tr = {
        .tx_buf = (uint64_t)(uintptr_t)spi_buffer,
        .rx_buf = 0,
        .len = CHUNK_SIZE,
        .speed_hz = speed,
        .bits_per_word = bits,
        .cs_change = 0,
    };
};

```

```

while (1) {
    int n = read(pipe_fd, recv_buf + recv_len, RING_BUFFER_SIZE
        - recv_len);
    if (n < 0) { usleep(1000); continue; }
    if (n == 0) break;
    recv_len += n;

    while (recv_len > 4) {
        int start_idx = -1, end_idx = -1;
        for (size_t i = 0; i < recv_len - 1; i++) {
            if (recv_buf[i] == 0xFF && recv_buf[i+1] == 0xD8) {
                start_idx = i; break; }
        }
        if (start_idx == -1) { recv_len = 0; break; }

        for (size_t i = start_idx; i < recv_len - 1; i++) {
            if (recv_buf[i] == 0xFF && recv_buf[i+1] == 0xD9) {
                end_idx = i + 1; break; }
        }

        if (start_idx != -1 && end_idx != -1) {
            size_t jpeg_size = end_idx - start_idx + 1;
            uint8_t *frame_buffer = malloc(jpeg_size);
            memcpy(frame_buffer, recv_buf + start_idx,
                jpeg_size);

            size_t remaining = recv_len - (end_idx + 1);
            memmove(recv_buf, recv_buf + end_idx + 1, remaining
                );
            recv_len = remaining;

            size_t bytes_sent = 0;
            int frame_corrupted = 0;

            while (bytes_sent < jpeg_size) {
                size_t to_send = jpeg_size - bytes_sent;
                if (to_send > CHUNK_SIZE) to_send = CHUNK_SIZE;
                memcpy(spi_buffer, frame_buffer + bytes_sent,
                    to_send);

                if (to_send < CHUNK_SIZE) {
                    memset(spi_buffer + to_send, 0x00,
                        CHUNK_SIZE - to_send);
                }

                if (wait_for_stm32_ready()) {
                    if (ioctl(spi_fd, SPI_IOC_MESSAGE(1), &tr)
                        < 1) {

```

```

        frame_corrupted = 1;
        break;
    }
    int ack_timeout = 0;
    while ((* (gpio + 13) & (1 << GPIO_PIN)) !=
           0) {
        ack_timeout++;
        if (ack_timeout > 50000) break;
    }
    drop_counter = 0;
} else {
    drop_counter++;
    frame_corrupted = 1;
    break;
}
bytes_sent += to_send;
}
free(frame_buffer);
if (frame_corrupted) continue;
} else {
    break;
}
}
}
free(recv_buf);
pclose(cam_pipe);
close(spi_fd);
return 0;
}

```

5 Firmware MCU du transmetteur (STM32U5 / spi_interface.c)

```

/*
 * Copyright 2021-2023 Morse Micro
 *
 * SPDX-License-Identifier: Apache-2.0
 */

#include <string.h>
#include <endian.h>
#include "mmosal.h"
#include "mmwlan.h"
#include "mmconfig.h"

```

```
#include "mmipal.h"
#include "lwip/icmp.h"
#include "lwip/tcpip.h"
#include "lwip/udp.h"
#include "lwip/netif.h"

#include "mm_app_common.h"
#include "stm32u5xx_hal.h"

// --- SPI ---
#define SPI_PAYLOAD_SIZE 1400

SPI_HandleTypeDef hspi1;
uint8_t spi_rx_buffer[SPI_PAYLOAD_SIZE];
volatile bool spi_packet_received = false;
// -----

// --- Variables de Profilage DWT ---
volatile uint32_t dwt_start = 0;
volatile uint32_t dwt_end = 0;
volatile uint32_t mcu_cycles = 0;
extern uint32_t SystemCoreClock; // Fourni par le STM32 (ex:
    160000000 pour 160 MHz)

static volatile bool is_network_ready = false;

/* Callback pour savoir quand la connexion est prete*/
static void link_status_callback(const struct mmipal_link_status *
    link_status)
{
    if (link_status->link_state == MMIPAL_LINK_UP) {
        printf("\n>>>_CONNECTE_A_OPENWRT_<<<\n");
        is_network_ready = true;
    }
}

static void udp_unicast_tx_start(struct udp_pcb *pcb)
{
    //err_t err;

    //ip_set_option(pcb, SOF_BROADCAST);
    ip_addr_t dest_ip;
    IP4_ADDR(ip_2_ip4(&dest_ip), 192, 168, 12, 10);
```

```
// --- ACTIVATION DU COMPTEUR DE CYCLES DWT ---
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

printf(">>>PONT_SPI-WIFI_ACTIVE!_En_attente_de_la_Rpi...<<<\n");
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
uint32_t packet_counter = 0;
while (1)
{
    if (spi_packet_received) {

        struct pbuf *p = pbuf_alloc(PBUF_TRANSPORT,
            SPI_PAYLOAD_SIZE, PBUF_REF);
        if (p != NULL) {
            p->payload = (void *)spi_rx_buffer;

            LOCK_TCPIP_CORE();
            udp_sendto(pcb, p, &dest_ip, 1337);
            UNLOCK_TCPIP_CORE();

            pbuf_free(p);
        }

        // --- ARRET DU CHRONO ---
        dwt_end = DWT->CYCCNT;
        mcu_cycles = dwt_end - dwt_start;

        // Affichage 1 fois tous les 100 paquets (pour ne
        pas bloquer le CPU avec l'UART)
        if ((packet_counter++ % 100) == 0) {
            // SystemCoreClock vaut 160000000 (160 MHz)
            uint32_t freq_mhz = SystemCoreClock / 1000000;
            uint32_t time_us = mcu_cycles / freq_mhz;

            printf("[Profilage]_Temps_de_traitement_MCU
                _:%lu_us_(%lu_cycles)\n", time_us,
                mcu_cycles);
        }

        spi_packet_received = false;

        // STM32 ecoute
        HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer,
            SPI_PAYLOAD_SIZE);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET)
        ;
    }
}
```



```

        } else {
            // mmosal_task_sleep(1);
        }
    }
}

/**
 * Initialize the UDP protocol control block. Binds to @ref
 *   DEFAULT_UDP_PORT
 *
 * @note If the parameters are set in the config store they will be
 *   used.
 *
 * @return Reference to the pcb is successfully initialized else
 *   NULL
 */
static struct udp_pcb *init_udp_pcb(void)
{
    struct udp_pcb *pcb = NULL;
    LOCK_TCPIP_CORE();
    pcb = udp_new();
    if (pcb != NULL) {
        udp_bind(pcb, IP_ANY_TYPE, 1337);
    }
    UNLOCK_TCPIP_CORE();
    return pcb;
}

void SPI_Slave_Init(void)
{
    // 1. Activer les horloges du SPI1, du Port E(SPI) et port D (
    //   Spare GPIO)
    __HAL_RCC_SPI1_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();

    // D10(PE12), D13(PE13), D12(PE14) et D11(PE15)
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
        GPIO_PIN_15;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;

    GPIO_InitStruct.Alternate = GPIO_AF5_SPI1; // Sur U5, Port E =
        SPI1 (AF5)
    HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);

```

```
// Config SPI1
hspi1.Instance = SPI1;
hspi1.Init.Mode = SPI_MODE_SLAVE;
hspi1.Init.Direction = SPI_DIRECTION_2LINES;
hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;

// CS sur D10
hspi1.Init.NSS = SPI_NSS_HARD_INPUT;

hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
hspi1.Init.TIMode = SPI_TIMODE_DISABLED;
hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLED;
hspi1.Init.CRCPolynomial = 7;

if (HAL_SPI_Init(&hspi1) != HAL_OK) {
    printf("ERREUR: Echec initialisation SPI1!\n");
} else {
    printf("SPI Esclave (SPI1) initialise sur PE12 a PE15!\n");
    ;
}

HAL_NVIC_SetPriority(SPI1_IRQn, 5, 0);
HAL_NVIC_EnableIRQ(SPI1_IRQn);

// Initialisation de la broche Handshake (PD15)
GPIO_InitTypeDef GPIO_InitStruct_Handshake = {0};
GPIO_InitStruct_Handshake.Pin = GPIO_PIN_15;
GPIO_InitStruct_Handshake.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct_Handshake.Pull = GPIO_NOPULL;
GPIO_InitStruct_Handshake.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct_Handshake);
}

// Quand le STM32 a reçu un paquet
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1) {
        // Demarrer le chrono
        dwt_start = DWT->CYCCNT;
        // mesure
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
        spi_packet_received = true;
    }
}

void SPI1_IRQHandler(void)
```

```
{
    HAL_SPI_IRQHandler(&hspi1);
}

/**
 * Main entry point to the application. This will be invoked in a
 * thread once operating system
 * and hardware initialization has completed. It may return, but it
 * does not have to.
 */
void app_init(void)
{
    printf("\n\n---RPI->STM32->HALOW---\n\n");

    SPI_Slave_Init();

    HAL_SPI_Receive_IT(&hspi1, spi_rx_buffer, SPI_PAYLOAD_SIZE);

    // low QoS config
    struct mmwlan_qos_queue_params fpv_qos = {
        .aci = 3,           // ACI 3 = Voice (TOS 0xC0 - LwIP)
        .aifs = 2,         // inter-trame waiting time
        .cw_min = 1,
        .cw_max = 1,
        .txop_max_us = 0
    };

    mmwlan_set_default_qos_queue_params(&fpv_qos, 1);
    mmwlan_set_power_save_mode(MMWLAN_PS_DISABLED);

    app_wlan_init();

    //mmwlan_override_max_tx_power(26);

    mmipal_set_link_status_callback(link_status_callback);

    printf("Connexion a l'AP OpenWrt en cours...\n");

    app_wlan_start();

    mmwlan_ate_override_rate_control(MMWLAN_MCS_1, MMWLAN_BW_8MHZ,
        MMWLAN_GI_NONE);
    printf("forage OK: 8MHz / MCS 1 force.\n");

    while (!is_network_ready) {
        mmosal_task_sleep(10);
    }
}
```

```
}  
  
struct udp_pcb *pcb = init_udp_pcb();  
if (pcb != NULL) {  
    pcb->tos = 0xC0;  
    udp_unicast_tx_start(pcb);  
}  
}
```

6 Système de transmission alternatif DVB-T2 (SDR / BladeRF)

Dans le cadre d'une comparaison, un système de transmission basé sur la norme de télévision numérique **DVB-T2** a été évalué. L'objectif était de comparer le Wi-Fi HaLow à un système utilisant le protocole industriel de diffusion par flux MPEG Transport Stream (MPEG-TS) via une SDR.

6.1 Matériel et architecture

L'architecture de test DVB-T2 comprend :

- **Émetteur (TX)** : Une Raspberry Pi 4B connectée en USB 3.0 à une SDR **BladeRF xA9** assurant la capture et l'encapsulation MPEG-TS.
- **Récepteur (RX)** : Un PC Windows équipé d'un dongle récepteur DVB-T2 USB standard (DVB Adapter 2).

6.2 Configuration et pipeline de l'émetteur (Raspberry Pi)

Le chargement du FPGA de la BladeRF s'effectue via les commandes :

```
sudo bladeRF-cli -L hostedxA9-latest.rbf  
bladeRF-cli -p  
# puis  
sudo bladeRF-cli -L output.rbf # fournit par un ingénieur de l'  
IICT
```

Le flux vidéo issu de la caméra est encodé en H.264 (profil *baseline*, option *zerolatency*), multiplexé en MPEG-TS, puis injecté directement via l'entrée standard (*stdin*) dans l'utilitaire *bladeRF-cli* sur le cœur isolé 3 :

```
sudo taskset -c 3 nice -n -20 gst-launch-1.0 -q \
  libcamerasrc ! \
  video/x-raw,width=640,height=480,framerate=30/1 ! \
  videoconvert ! queue max-size-buffers=2 leaky=downstream ! \
  x264enc tune=zerolatency speed-preset=ultravast key-int-max=30
  bitrate=1500 ! \
  video/x-h264,profile=baseline ! \
  mpegtsmux alignment=7 ! \
  fdsink fd=1 sync=false | \
sudo bladeRF-cli -e "set_samplerate_tx1_20M" -e "set_bandwidth_tx1_
8M" \
-e "set_frequency_tx1_634M" -e "set_gain_tx1_60" \
-e "set_samplerate_tx2_20M" -e "set_bandwidth_tx2_8M" \
-e "set_frequency_tx2_634M" -e "set_gain_tx2_60" \
-e "tx_config_timeout=80000" \
-e "tx_config_file=/dev/stdin format=bin channel=1,2" \
-e "tx_start" -e "tx_wait"
```

Note : Le format MJPEG n'a pas été exploité, l'encapsulation MPEG-TS ne prenant pas en charge nativement ce codec.

6.3 Configuration du récepteur et décodage (PC Windows)

Deux méthodes d'acquisition ont été expérimentées sur la station de réception afin d'analyser l'impact des couches logicielles sur la latence du flux MPEG-TS.

6.3.1 Approche standard : VLC Media Player

La première solution s'appuie sur VLC pour s'accorder sur la fréquence UHF de 634 MHz, extraire le flux réseau démodulé et le renvoyer en UDP local vers GStreamer :

```
# Lancement de l'acquisition DVB-T2 et encapsulation vers le port
local
"C:\Program Files (x86)\VideoLAN\VLC\vlc.exe" "dvb-t2://frequency
=634000000:bandwidth=8" :dvb-adapter=2 :dvb-a-modulation=QPSK :
dvb-plp-id=0 :live-caching=50 --sout="#std{access=udp,mux=ts,dst
=127.0.0.1:1234}" --sout-mux-caching=50 --network-caching=50 -I
dummy
```

6.3.2 Approche secondaire : TSDuck

Pour s'affranchir des tampons internes importants de VLC, l'outil d'analyse de flux MPEG **TSDuck** a été configuré pour assurer une réception directe et plus brute :

```
# Capture directe du canal physique et redirection brute en UDP via  
TSDuck  
tsp.exe -v -I dvb --adapter 2 --delivery-system DVB-T --frequency  
634000000 --bandwidth 8000000 --modulation QPSK --transmission-  
mode auto --guard-interval auto --high-priority-fec auto --  
signal-timeout 10 -O ip 127.0.0.1:1234
```

6.3.3 Pipeline d'affichage GStreamer (Commun aux deux méthodes)

Indépendamment de l'outil d'acquisition amont (VLC ou TSDuck), le flux de transport MPEG-TS démodulé et réceptionné sur le port local 1234 est extrait, décodé matériellement et affiché par GStreamer via Direct3D 11 :

```
gst-launch-1.0.exe -v udpsrc port=1234 caps="video/mpegts" !  
tsdemux ! queue max-size-buffers=2 max-size-bytes=0 max-size-  
time=0 leaky=downstream ! h264parse config-interval=-1 !  
avdec_h264 max-threads=1 ! videoconvert ! d3d11videosink sync=  
false
```

6.4 Bilan des mesures

Bien que la modulation QPSK et la faible fréquence (630Mhz) offre une excellente immunité face aux perturbations, la latence mesurée s'est avérée incompatible avec le pilotage FPV, s'établissant entre 2 et 7 secondes. L'ajustement à la baisse des paramètres de mise en mémoire tampon (*caching*) permet de stabiliser la latence autour de 2 secondes.